

IT 2012 - Unstructured Data

Lab 10 - Data Analysis and Aggregation

Assist. Prof. Dr. Dželila Mehanović

Introduction

In Lab 9 you built a data cleaning pipeline. You handled missing values, fixed string columns, removed duplicates, converted data types, and validated your cleaned dataset. In Lab 10 you will use that clean data to perform real analytical work. Data analysis and aggregation means asking structured questions about your data and computing answers. You will combine data from MySQL and MongoDB, reshape DataFrames, group and aggregate values, build pivot tables, and work with dates and time series. By the end of this lab you will have a complete analytical notebook that answers key questions about the movie industry.

Note: This lab continues directly from Lab 9 so make sure your `processed/cleaned/cleaned_data.csv` file exists before you start.

Learning Objectives

By the end of this lab you will be able to:

- Connect to a MySQL database from Python using PyMySQL
- Query data from MySQL into a pandas DataFrame using `pd.read_sql()`
- Combine data from MySQL and MongoDB into a single DataFrame
- Merge multiple DataFrames using inner, left, right, and outer joins
- Reshape data between wide and long formats using `pivot()` and `melt()`
- Group data and compute aggregations using `groupby()` and `agg()`
- Apply multiple aggregation functions to different columns simultaneously
- Build multi-dimensional summaries using `pivot_table()` and `crosstab()`
- Parse date columns and extract components such as year, month, and weekday
- Resample time series data to different frequencies
- Compute rolling averages and other moving window statistics
- Build a MongoDB aggregation pipeline with `$match`, `$group`, `$sort`, and `$project`
- Answer specific analytical questions about your movie dataset
- Document findings and insights clearly

This lab introduces a new `src/analytics/` package structure with dedicated modules for each analysis type, a new `notebooks/lab10_data_analysis.ipynb` notebook, and updated `run_pipeline.py` integration. This lab adds new files to your existing project.

Inside the `data/processed/analytics/` folder, create a new directory where all generated analytics results will be stored. In this folder, you will produce three CSV files. For example, in a movie analytics project, one file (`genre_analysis.csv`) will store aggregated insights about movie genres, `yearly_trends.csv` will analyze how data changes over time (e.g., number of movies per year), and `pivot_genre_year.csv` will contain a pivot table combining genres and years for easier

comparison and visualization. For your own projects, you should adjust these themes and file contents to match your specific topic.

Within the **src/analytics/** module, extend the existing structure by adding several new Python files, each responsible for a specific part of the analytics pipeline. Create **db_connector.py** to handle the connection to a MySQL database, which will be used if you need to fetch or store data externally. Add **data_combiner.py** to implement logic for merging or concatenating multiple datasets into one unified DataFrame. Then, create an **aggregator.py** where you will perform grouping and aggregation operations (such as averages, counts, sums) on the dataset.

Next, implement **pivot_builder.py**, which will be responsible for creating pivot tables from your data to summarize it in a more structured way. Add **time_series.py** to handle date-related operations, such as converting columns to datetime format and resampling data (e.g., yearly or monthly trends). Create **mongo_pipeline.py** to define and execute MongoDB aggregation pipelines in case you are working with NoSQL data sources. Finally, add **insight_reporter.py**, which will generate a summary of key insights from your analysis, potentially printing or saving them in a readable format.

In the **notebooks/** folder, create a new Jupyter notebook named **lab10_data_analysis.ipynb**. This notebook will serve as your main submission file, where you will demonstrate the full analytics workflow step by step, including loading data, applying transformations, and presenting results. Update the **run_pipeline.py** script to include a new step that executes all the analytics logic you implemented in the new modules. This ensures the pipeline runs end-to-end, including both preprocessing and analytics.

Lastly, update the **requirements.txt** file by adding any new dependencies needed for this lab, such as database connectors (e.g., mysql-connector-python, pymongo) or additional data processing libraries.

Part 1 - Install Required Libraries and Tools

Open **requirements.txt** and add the following lines if they are not already present:

```
pymysql>=1.1.0
sqlalchemy>=2.0.0
seaborn>=0.12.0
```

Then activate your virtual environment in the VS Code terminal and install everything:

```
pip install -r requirements.txt
```

1.1 Set Up MySQL

This lab requires a MySQL database. If you do not already have MySQL installed, follow these steps. If MySQL is already installed then skip to the database creation step.

Install MySQL on your machine. For Windows, download MySQL Community Server from mysql.com. For macOS, brew install mysql. For Ubuntu, run `sudo apt install mysql-server`.

Start the MySQL service and log in:

```
# macOS / Linux
mysql -u root -p

# Windows (MySQL Shell or Command Prompt)
mysql -u root -p
```

Create a database and a table for your project.

1.2 Populate MySQL from Your Cleaned CSV

Create `src/analytics/db_connector.py` to connect the cleaned movie dataset with a MySQL database and populate the table you created in the previous step. The file should include clear, reusable functions for opening a PyMySQL connection, inserting financial columns such as `tmdb_id`, `title`, `budget`, `revenue`, `release_year`, and `genres`, skipping invalid rows, and querying the table back into a Pandas DataFrame using `pd.read_sql()`.

In the notebook, load the cleaned CSV, create a database connection, call `populate_financials()` to insert the cleaned records, and then use `query_financials()` to verify that the data was successfully stored. Make sure SQL filtering is done with WHERE clauses, inserts use parameterized queries, and the MySQL connection is closed after the work is finished.

Part 2 - Combining Data from MySQL and MongoDB

Real-world data is rarely stored in one place. Your pipeline pulls metadata from API (stored in MongoDB), but financial details such as budget and revenue may live in a relational database. Combining these sources gives you a richer dataset for analysis.

pandas gives you three tools for combining DataFrames. The right choice depends on where your joining key lives:

Function	When to use it	Example
<code>pd.concat()</code>	Stack tables with the same columns (append rows)	Combine monthly files into one year
<code>pd.merge()</code>	Join on a shared key column (SQL-style join)	Merge MongoDB metadata with MySQL financials on <code>tmdb_id</code>

<code>df.join()</code>	Join on the index (index-based merge)	Combine two DataFrames that share the same index
------------------------	---------------------------------------	--

Create **src/analytics/data_combiner.py** to combine financial data from MySQL with movie metadata from MongoDB or the cleaned CSV. The file should include reusable functions for merging two DataFrames on `tmdb_id`, comparing different join types, and concatenating DataFrames when they share the same structure. In **notebooks/lab10_data_analysis.ipynb**, add cells to import the analytics functions, load the cleaned dataset, populate MySQL, query financial rows with valid budget and revenue, create a MongoDB-style metadata DataFrame, and merge both sources into one combined DataFrame. Finally, demonstrate inner, left, right, and outer joins, print their row counts, and save a bar chart showing how each join type affects the final dataset size.

[Part 3 - Reshaping Data with `pivot\(\)` and `melt\(\)`](#)

3.1 Wide vs Long Format

Most raw data files are in wide format: each row represents one movie, and separate columns hold each metric such as budget, revenue, and `vote_average`. However, some analysis and most plotting libraries expect long format, where each row represents one single measurement.

Format	Structure	Best for
Wide	One row per movie, one column per metric	Compact storage, human reading
Long	One row per measurement, a column names the metric	Plotting with seaborn/matplotlib, databases

Create **src/analytics/pivot_builder.py** to add reusable functions for reshaping and summarizing the combined movie dataset. The module should support converting wide data to long format with `melt()`, converting long data back to wide format with `pivot()`, building aggregated pivot tables, and creating cross-tabulation tables. In **notebooks/lab10_data_analysis.ipynb**, add cells to extract a `primary_genre` column, reshape financial columns such as budget, revenue, and popularity into long format, and build a pivot table showing average revenue by `release_year` and `genre`. Finally, create a language-versus-decade cross-tabulation and save the `genre/year` pivot table as a CSV file for later analysis.

[Part 4 - GroupBy Analysis](#)

GroupBy follows three steps:

1. split the DataFrame into groups,
2. apply a function to each group independently,
3. then combine the results back into a single object.

Calling `df.groupby()` returns a GroupBy object and nothing is computed until you call `.mean()`, `.sum()`, or `.agg()`.

Create **`src/analytics/aggregator.py`** to add reusable aggregation functions for analyzing the cleaned movie dataset. The module should calculate genre-level summaries, yearly trends, and top N movies per group based on columns such as `primary_genre`, `vote_average`, `revenue_usd`, and `budget_usd`. In **`notebooks/lab10_data_analysis.ipynb`**, add cells to generate a genre summary, save it as `genre_analysis.csv`, calculate yearly trends from 1990 to 2024, visualize movie count and total revenue over time, and save the chart as `yearly_trends.png`. Finally, use `top_n_per_group()` to display the top 3 movies per genre by revenue so the highest-performing films can be compared within each category.

Method	Use case
<code>agg("mean")</code>	Single aggregation on all numeric columns
<code>agg(["mean","std"])</code>	Multiple aggregations on all numeric columns
<code>agg(name=("col","func"))</code>	Named aggregation - gives clean column names
<code>transform(func)</code>	Return same-shape result - broadcast group stats back to rows
<code>filter(lambda x: len(x)>5)</code>	Drop entire groups that fail a condition
<code>apply(func)</code>	Most flexible - returns scalar, Series, or DataFrame

[Part 5 - Time Series Analysis](#)

A `DatetimeIndex` is the foundation of time series work in pandas. Once a column is parsed as `datetime64`, you can slice by year, resample to any frequency, and compute rolling statistics.

Create **`src/analytics/time_series.py`** to add reusable functions for time-based movie analysis. The module should parse `release_date`, extract date components such as year, month, weekday, and quarter, build monthly revenue time series, resample data into yearly or quarterly totals, and calculate rolling averages. In **`notebooks/lab10_data_analysis.ipynb`**, add cells to import these functions, parse release dates from the cleaned dataset, display valid and missing date counts, generate monthly and yearly `revenue_usd` trends, and calculate 3-month, 6-month, and 12-month

rolling averages. Finally, visualize monthly revenue with rolling averages and save the chart as `rolling_revenue.png` for later analysis.

[Part 6 - MongoDB Aggregation Pipeline](#)

MongoDB aggregation pipelines process documents server-side before returning them to Python. A pipeline is a list of stage dicts, each transforming the documents passing through it.

Stage	Purpose
\$match	Filter documents before processing (like SQL WHERE)
\$addFields / \$unwind	Split genre string into array and expand one doc per genre
\$group	Aggregate: count, sum, avg per genre
\$sort	Order results (descending by avg_rating)
\$project	Select and rename output fields (like SQL SELECT)

[Part 7 - Insight Reporter and Analytical Questions](#)

Create `src/analytics/insight_reporter.py` to answer the main analytical questions from the cleaned and combined movie dataset. The module should include functions for finding top genres by average revenue, calculating ROI by genre, tracking yearly movie release volume, and summarizing the distribution of movies by original language.

In [notebooks/lab10_data_analysis.ipynb](#), add cells to prepare `primary_genre`, merge missing financial columns if needed, and run `run_all_questions()` to print a formatted findings summary. Finally, visualize the top 10 genres by ROI, label each bar with its return value, and save the chart as `genre_roi.png`.

[Part 8 - Integrate Analytics into run_pipeline.py](#)

Update the existing `run_pipeline.py` so it includes a full Lab 10 analytics pipeline after the cleaning step.

The pipeline should load the cleaned file from `data/processed/cleaned/movies_clean.csv`, connect to MySQL, populate and query financial data, parse release dates, create a `primary_genre` column, merge financial data back into the main dataset, and generate analytics outputs. It should save files such as `genre_analysis.csv` and `yearly_trends.csv` inside `data/processed/analytics`, run the analytical questions summary, and use logging to record each major step. The script should also check whether the cleaned CSV exists first, run the cleaning pipeline if needed, and then execute the analytics pipeline automatically.

Assignment Tasks:

Apply all parts of this lab to your own project. Column names must match your `movies_clean.csv`.

- MySQL Connection (10%) - Connect to the movies database, create the `movie_financials` table with columns and populate it from `movies_clean.csv`, and query it back using `pd.read_sql()`.
- Combining Data Sources (10%) - Merge at least two DataFrames using different join types. Show row counts for all four join types and explain which is most appropriate.
- Data Reshaping (10%) - Use `melt()` to convert at least three numeric columns (`budget_usd`, `revenue_usd`, `popularity`) from wide to long format. Use `pivot_table()` to create a two-dimensional summary.
- GroupBy Analysis (15%) - Compute a grouped summary using at least four aggregation functions (mean, sum, count, median). Use named aggregations for clean column names. Find the top N items per group using `apply()`.
- Pivot Tables (10%) - Build a pivot table showing `revenue_usd` broken down by `release_year` and `primary_genre`. Add `margins=True` to include subtotals.
- Time Series (15%) - Parse `release_date` to `datetime64`. Extract at least three date components. Build a monthly time series. Resample to yearly. Compute rolling averages with window sizes 3, 6, and 12.
- MongoDB Aggregation Pipeline (10%) - Write a pipeline with at least four stages: `$match`, `$group`, `$sort`, and `$project` or `$unwind`. Convert the result to a DataFrame.
- Analytical Questions (15%) - Answer at least four questions, each with a quantified finding, the code that produced it, and a chart saved as PNG.
- Pipeline Integration (5%) - Add the analytics step to `run_pipeline.py` so the full pipeline runs with a single command.